

Who First? — Early Warning Analysis — CDMX

Adaptación de `who_first.ipynb` (Mauro et al., 2025) para Ciudad de México.

Responde: **¿Cuál métrica detecta la gentrificación primero: G_bin o G_net?** Genera Fig. 4c del paper y las Notas Suplementarias 1-3.

Análisis incluidos:

1. Cross-correlación entre señales G_bin y G_net (con baseline shuffle)
2. Información mutua entre señales G_bin y G_net
3. Visualización de barcodes (eventos discretos)

Cambios respecto al original:

- Grid 5x5, rutas con `mode='improve'` y `starting_deployment='centre_sgr-big'`
- `from ABM_MeanField_Cells.model import GentModel` → eliminado (no se usa)
- Columna de ejemplo `'(5, 6)'` → `'(2, 3)'` (válida para grilla 5x5)
- `width=height=5` en el setup

```
In [1]: # CAMBIO: eliminado 'from ABM_MeanField_Cells.model import GentModel' (no se
from ABM_MeanField_Cells.utils import *
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm.notebook import tqdm
from utils_gent_measure import *
import networkx as nx
import matplotlib.gridspec as gridspec
import math
import os

from functools import partial
from concurrent.futures import ProcessPoolExecutor, as_completed
from collections import defaultdict
from sklearn.metrics import mutual_info_score

np.set_printoptions(linewidth=400)
%matplotlib inline

pd.options.mode.chained_assignment = None
sns.set()
sns.set(rc={'figure.figsize': (16, 9)})
sns.set_style("whitegrid")
```

```
sns.set_context("talk")

import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
warnings.simplefilter(action='ignore', category=pd.errors.PerformanceWarning)

# CAMBIO: grid 5x5
width = height = 5

import colorcet as cc
palette = sns.color_palette(cc.glasbey, n_colors=width * height)
```

```
In [2]: # — CDMX: rutas con mode='improve' y 'centre_segr-big' —————
mode = "improve"
starting_deployment = "centre_segr-big"
num_agents = 2**12

directory = f"out/batch_results/{mode}/{starting_deployment}/{num_agents}"
intermediate_dir = os.path.join(directory, "intermediate")
exp_directory = os.path.join(directory, "exps")

out_plot_directory = os.path.join("out/plots", starting_deployment)
os.makedirs(f"{out_plot_directory}/{num_agents}", exist_ok=True)

print("intermediate:", intermediate_dir)
print("exps:      ", exp_directory)
```

```
intermediate: out/batch_results/improve/centre_segr-big/4096agents/intermediate
exps:      out/batch_results/improve/centre_segr-big/4096agents/exps
```

1. Funciones auxiliares (binarización y correlación)

```
In [3]: def create_chi_hat_bin(chi_hat_series):
    result = np.zeros_like(chi_hat_series)
    changes = np.diff(chi_hat_series)
    shift_indices = np.where(changes == 1)[0] + 1
    result[shift_indices] = 1
    return result

def create_gamma_bin(gamma_series, find_peaks_fn):
    result = np.zeros_like(gamma_series)
    peak_indices = find_peaks_fn(gamma_series)
    peak_indices = np.round(peak_indices).astype(int)
    peak_indices = peak_indices[(peak_indices >= 0) & (peak_indices < len(result))]
    result[peak_indices] = 1
    return result

def calculate_cross_correlation(chi_hat_bin, gamma_bin, max_lag=10):
    correlation_values = []
    lags = range(-max_lag, max_lag + 1)
    for lag in lags:
        if lag < 0:
            s1, s2 = chi_hat_bin[-lag:], gamma_bin[:lag]
        elif lag > 0:
            s1, s2 = chi_hat_bin[:lag], gamma_bin[-lag:]
        else:
            s1, s2 = chi_hat_bin, gamma_bin
        correlation_values.append(np.corrcoef(s1, s2)[0, 1])
    return correlation_values
```

```

        s1, s2 = chi_hat_bin[:-lag], gamma_bin[lag:]
    else:
        s1, s2 = chi_hat_bin, gamma_bin
    corr = np.corrcoef(s1, s2)[0, 1] if (np.any(s1) and np.any(s2)) else 0
    correlation_values.append(corr)
return lags, correlation_values

def calculate_shuffled_correlations_bidirectional(gamma_bin, chi_hat_bin, num_shuffles):
    shuffled_chi = [np.random.permutation(chi_hat_bin) for _ in range(num_shuffles)]
    shuffled_gamma = [np.random.permutation(gamma_bin) for _ in range(num_shuffles)]
    all_corrs = []
    for i in range(num_shuffles):
        _, neg = calculate_cross_correlation(shuffled_chi[i], gamma_bin, max_lag)
        _, pos = calculate_cross_correlation(chi_hat_bin, shuffled_gamma[i], max_lag)
        combined = list(neg[:max_lag + 1]) + list(pos[max_lag + 1:])
        all_corrs.append(combined)
    all_corrs = np.array(all_corrs)
    return np.mean(all_corrs, 0), np.percentile(all_corrs, 5, 0), np.percentile(all_corrs, 95, 0)

```

2. Cross-correlación — todos los p_g (Fig. 4c del paper)

Este bloque tarda ~5-15 min en el M5 con `ProcessPoolExecutor`.

`pg_values = [0.01, 0.05, 0.1, 0.15]` — todos disponibles en los datos CDMX.

```

In [4]: h = 20
delta = 15
max_lag = delta
num_rep = 150

# Todos los p_g disponibles (excluimos 0.0 porque sin movimiento H no hay señal)
pg_values = [0.01, 0.05, 0.1, 0.15]
intermediate_folder = intermediate_dir

def process_file_pair(chi_hat_file, gamma_file, h, max_lag):
    result, shuffled_result, lags = None, None, None
    if os.path.exists(chi_hat_file) and os.path.exists(gamma_file):
        chi_hat_df = pd.read_csv(chi_hat_file, index_col=0)
        gamma_df = pd.read_csv(gamma_file, index_col=0)
        chi_hat_df = chi_hat_df.fillna({col: 0 for i in range(h+1)} for col in chi_hat_df.columns)
        gamma_df = gamma_df.fillna({col: 0 for i in range(h+1)} for col in gamma_df.columns)
        all_lc, all_sm, all_sl, all_su = [], [], [], []
        for col in chi_hat_df.columns:
            if col in gamma_df.columns:
                chi_bin = create_chi_hat_bin(chi_hat_df[col].values)
                gam_bin = create_gamma_bin(gamma_df[col].values, find_peaks=True)
                cv = calculate_cross_correlation(chi_bin, gam_bin, max_lag)
                all_lc.append(cv)
                sm, sl, su = calculate_shuffled_correlations_bidirectional(chi_hat_df[col].values, gam_bin, num_rep)
                all_sm.append(sm); all_sl.append(sl); all_su.append(su)
        if all_lc:

```

```

        result = np.mean(all_lc, 0)
        shuffled_result = (np.mean(all_sm, 0), np.mean(all_sl, 0), np.me
    return lags, result, shuffled_result

```

In [5]: all_correlations, all_shuffled_means, all_shuffled_loweres, all_shuffled_upperes, lags_list = []

```

for p_g in pg_values:
    chi_files = [f"{intermediate_folder}/pg_{p_g}_h_{h}_rep_{r}_delta_{delta}"
    gam_files = [f"{intermediate_folder}/pg_{p_g}_h_{h}_rep_{r}_delta_{delta}"

    alc, alsm, als_l, als_u = [], [], [], []

    # CAMBIO: loop secuencial - ProcessPoolExecutor no funciona en Jupyter/n
    for cf, gf in tqdm(zip(chi_files, gam_files), total=len(chi_files), desc=
        lgs, res, shuf = process_file_pair(cf, gf, h, max_lag)
        if lgs is not None: lags_list.append(lgs)
        if res is not None: alc.append(res)
        if shuf is not None:
            alsm.append(shuf[0]); als_l.append(shuf[1]); als_u.append(shuf[2])

    if alc: all_correlations.append(np.mean(alc, 0))
    if alsm: all_shuffled_means.append(np.mean(alsm, 0))
    if als_l: all_shuffled_loweres.append(np.mean(als_l, 0))
    if als_u: all_shuffled_upperes.append(np.mean(als_u, 0))

lags = lags_list[0] if lags_list else list(range(-max_lag, max_lag + 1))
print("Listo. p_g procesados:", pg_values)

```

```

p_g=0.01: 0%|          | 0/150 [00:00<?, ?it/s]
p_g=0.05: 0%|          | 0/150 [00:00<?, ?it/s]
p_g=0.1: 0%|          | 0/150 [00:00<?, ?it/s]
p_g=0.15: 0%|          | 0/150 [00:00<?, ?it/s]
Listo. p_g procesados: [0.01, 0.05, 0.1, 0.15]

```

In [6]: # — Figura: 2x2 subplots, uno por p_g —

```

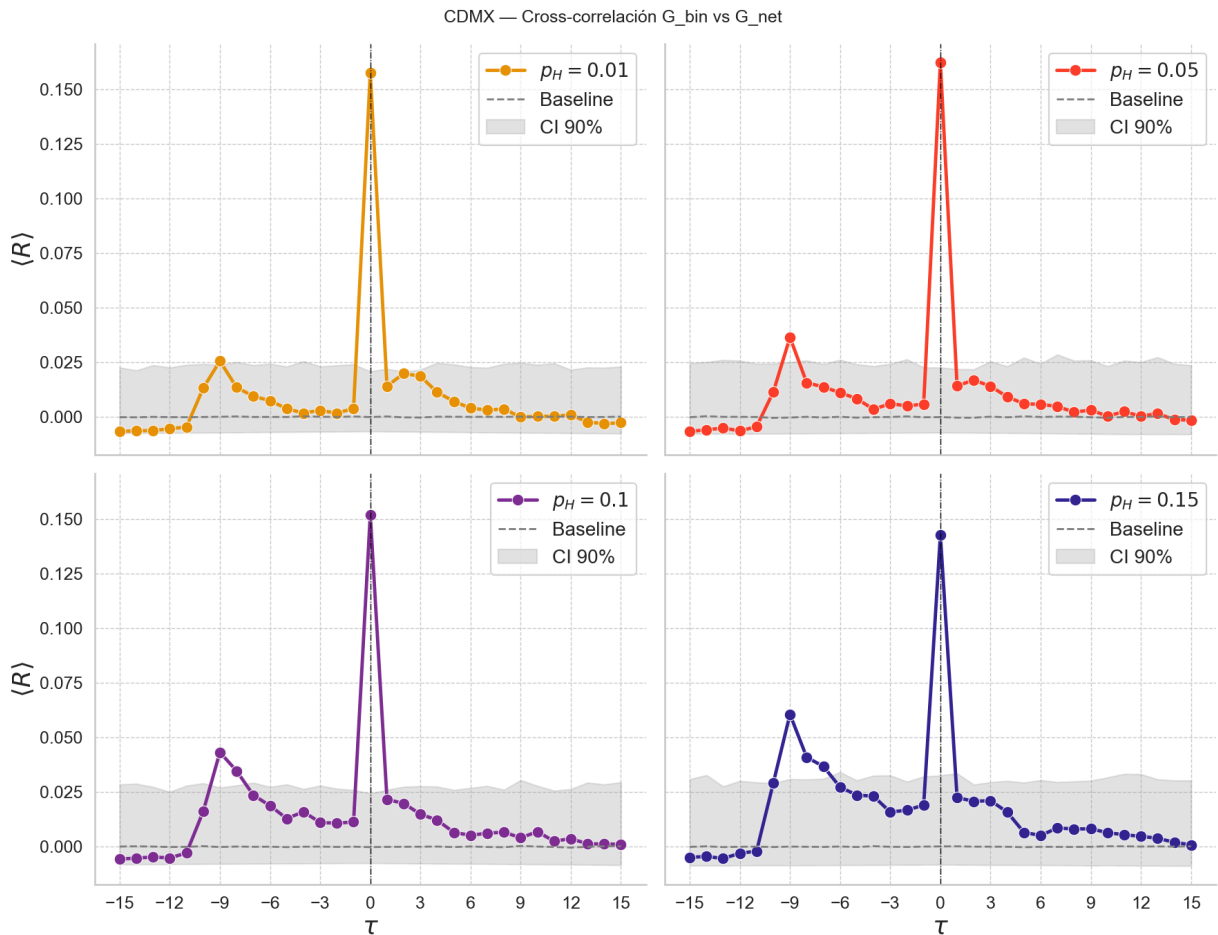
palette_corr = sns.color_palette("CMRmap_r", len(pg_values) + 1)[1:]

fig, axes = plt.subplots(2, 2, figsize=(18, 14), sharex='col', sharey=True)
axes = axes.flatten()

for i, pg in enumerate(pg_values):
    ax = axes[i]
    sns.lineplot(x=lags, y=all_correlations[i], marker='o', label=f'$p_H={pg}$',
        color=palette_corr[i], linewidth=3.5, markersize=12, ax=ax)
    ax.plot(lags, all_shuffled_means[i], color='gray', linestyle='--', linewidth=1)
    ax.fill_between(lags, all_shuffled_loweres[i], all_shuffled_upperes[i], color='gray')
    if i % 2 == 0: ax.set_ylabel(r'$\langle R \rangle$', fontsize=26)
    if i >= 2: ax.set_xlabel(r'$\tau$', fontsize=26)
    ax.set_xticks(np.arange(-max_lag, max_lag + 1, 3))
    ax.tick_params(axis='both', labelsize=18)
    ax.axvline(0, color='black', linestyle='-.', linewidth=1)
    ax.legend(fontsize=20, loc='upper right')
    ax.grid(True, linestyle='--', linewidth=1)
    sns.despine(ax=ax)

```

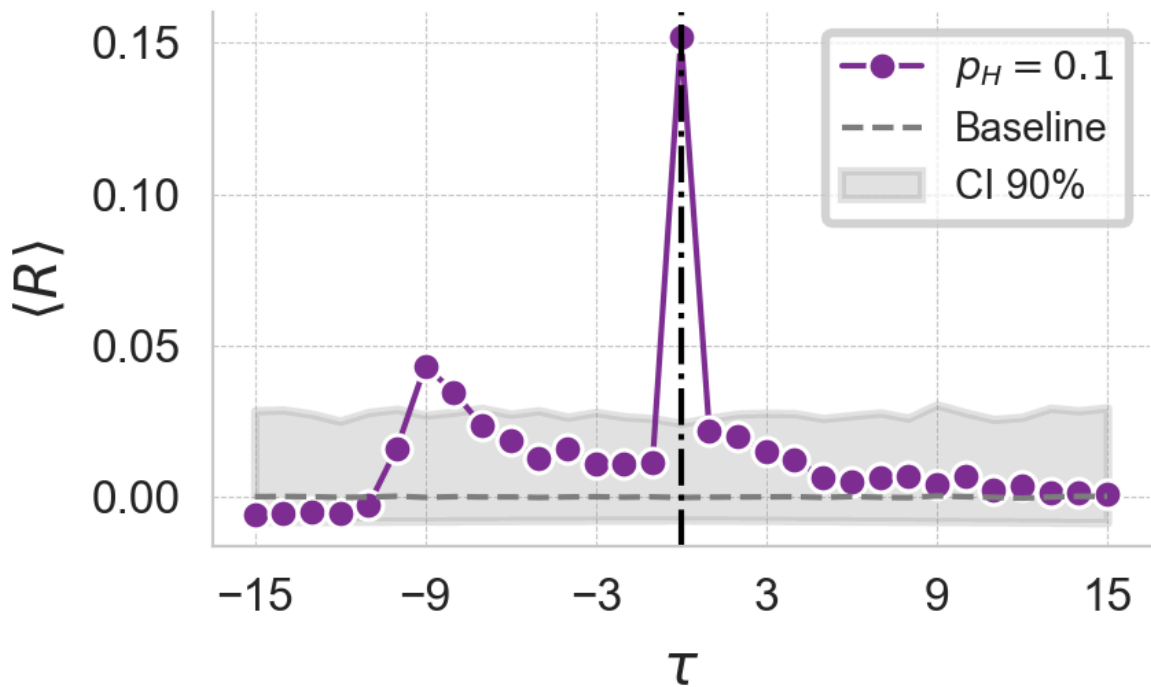
```
plt.suptitle("CDMX - Cross-correlación G_bin vs G_net", fontsize=18)
plt.tight_layout()
plt.savefig(f"{out_plot_directory}/{num_agents}/cdmx_ALL_cross_correlation.p
plt.show()
```



```
In [7]: # — Versión compacta para paper (un solo p_g)
# Elige el índice del p_g que quieres mostrar
idx_pg = pg_values.index(0.1) # cambia a 0.01, 0.05, 0.15 si prefieres

fig, ax = plt.subplots(figsize=(3.3, 2.2), dpi=300)
sns.lineplot(x=lags, y=all_correlations[idx_pg], marker='o',
             label=f'$p_H={pg_values[idx_pg]}$', color=palette_corr[2],
             linewidth=1, markersize=5, ax=ax)
ax.plot(lags, all_shuffled_means[idx_pg], color='gray', linestyle='--', line
ax.fill_between(lags, all_shuffled_lowers[idx_pg], all_shuffled_uppers[idx_pg],
               color='gray', alpha=0.2, label='CI 90%')
ax.set_ylabel(r'$\angle R \angle$', fontsize=10)
ax.set_xlabel(r'$\tau$', fontsize=10)
ax.set_xticks(np.arange(-max_lag, max_lag + 1, 6))
ax.tick_params(axis='both', labelsiz=8, length=2, width=1)
ax.axvline(0, color='black', linestyle='-.', linewidth=1)
ax.grid(True, linestyle='--', linewidth=0.25)
sns.despine(ax=ax)
ax.spines['left'].set_linewidth(0.5)
ax.spines['bottom'].set_linewidth(0.5)
ax.legend(fontsize=7, loc='upper right')
plt.tight_layout()
```

```
plt.savefig(f"{out_plot_directory}/{num_agents}/cdmx_cross_correlation_pg{pg}")
plt.show()
```



3. Información Mutua entre G_bin y G_net

(Suplementario del paper)

```
In [8]: def calculate_mutual_information(s1, s2, n_bins=2):
    c1 = np.floor(np.asarray(s1) * n_bins).astype(int)
    c2 = np.floor(np.asarray(s2) * n_bins).astype(int)
    return mutual_info_score(c1, c2)

def calculate_mutual_information_lags(chi_hat_bin, gamma_bin, max_lag=10):
    mi_vals, lags = [], range(-max_lag, max_lag + 1)
    for lag in lags:
        if lag < 0: s1, s2 = chi_hat_bin[-lag:], gamma_bin[:lag]
        elif lag > 0: s1, s2 = chi_hat_bin[:lag], gamma_bin[lag:]
        else: s1, s2 = chi_hat_bin, gamma_bin
        mi_vals.append(calculate_mutual_information(s1, s2) if (np.any(s1) & np.any(s2)) else 0)
    return lags, mi_vals

def calculate_shuffled_mutual_information_bidirectional(gamma_bin, chi_hat_bin, num_shuffles=100):
    shuf_chi = [np.random.permutation(chi_hat_bin) for _ in range(num_shuffles)]
    shuf_gam = [np.random.permutation(gamma_bin) for _ in range(num_shuffles)]
    all_mi = []
    for i in range(num_shuffles):
        _, neg = calculate_mutual_information_lags(shuf_chi[i], gamma_bin, max_lag=10)
        _, pos = calculate_mutual_information_lags(chi_hat_bin, shuf_gam[i], max_lag=10)
        all_mi.append(list(neg[:max_lag + 1]) + list(pos[:max_lag + 1]))
    all_mi = np.array(all_mi)
    return np.mean(all_mi, 0), np.percentile(all_mi, 5, 0), np.percentile(all_mi, 95, 0)
```

```
def process_files_mi(chi_hat_file, gamma_file, h, max_lag):
    if not (os.path.exists(chi_hat_file) and os.path.exists(gamma_file)):
        return None, [], [], [], []
    chi_df = pd.read_csv(chi_hat_file, index_col=0)
    gam_df = pd.read_csv(gamma_file, index_col=0)
    chi_df = chi_df.fillna({c: {i: 0 for i in range(h+1)} for c in chi_df.columns})
    gam_df = gam_df.fillna({c: {i: 0 for i in range(h+1)} for c in gam_df.columns})
    all_mi, all_sm, all_sl, all_su = [], [], [], []
    lags = None
    for col in chi_df.columns:
        if col in gam_df.columns:
            cb = create_chi_hat_bin(chi_df[col].values)
            gb = create_gamma_bin(gam_df[col].values, find_peaks_custom)
            lags, mv = calculate_mutual_information_lags(cb, gb, max_lag)
            all_mi.append(mv)
            sm, sl, su = calculate_shuffled_mutual_information_bidirectional(cb, gb, max_lag)
            all_sm.append(sm); all_sl.append(sl); all_su.append(su)
    return lags, all_mi, all_sm, all_sl, all_su
```

```
In [9]: all_mi_pg, all_mi_sm, all_mi_sl, all_mi_su = [], [], [], []
        lags_mi_list = []

        for p_g in pg_values:
            chi_files = [f"{intermediate_folder}/pg_{p_g}_h_{h}_rep_{r}_delta_{delta}"
                        for r in range(rep)]
            gam_files = [f"{intermediate_folder}/pg_{p_g}_h_{h}_rep_{r}_delta_{delta}"
                        for r in range(rep)]
            alm, alsm, alsl, alsu = [], [], [], []

            # CAMBIO: loop secuencial – ProcessPoolExecutor no funciona en Jupyter/
            for cf, gf in tqdm(zip(chi_files, gam_files), total=len(chi_files), desc=f"pg={p_g}"):
                lgs, lm, sm, sl, su = process_files_mi(cf, gf, h, max_lag)
                if lgs is not None: lags_mi_list.append(lgs)
                alm.extend(lm); alsm.extend(sm); alsl.extend(sl); alsu.extend(su)

            if alm: all_mi_pg.append(np.mean(alm, 0))
            if alsm: all_mi_sm.append(np.mean(alsm, 0))
            if alsl: all_mi_sl.append(np.mean(alsl, 0))
            if alsu: all_mi_su.append(np.mean(alsu, 0))

        lags_mi = lags_mi_list[0] if lags_mi_list else list(range(-max_lag, max_lag))
        print("MI calculada para:", pg_values)
```

```
MI p_g=0.01: 0%|          | 0/150 [00:00<?, ?it/s]
MI p_g=0.05: 0%|          | 0/150 [00:00<?, ?it/s]
MI p_g=0.1: 0%|           | 0/150 [00:00<?, ?it/s]
MI p_g=0.15: 0%|          | 0/150 [00:00<?, ?it/s]
MI calculada para: [0.01, 0.05, 0.1, 0.15]
```

```
In [10]: # — Figura compacta de Información Mutua —
        palette_mi = sns.color_palette("CMRmap_r", len(pg_values) + 1)[1:]

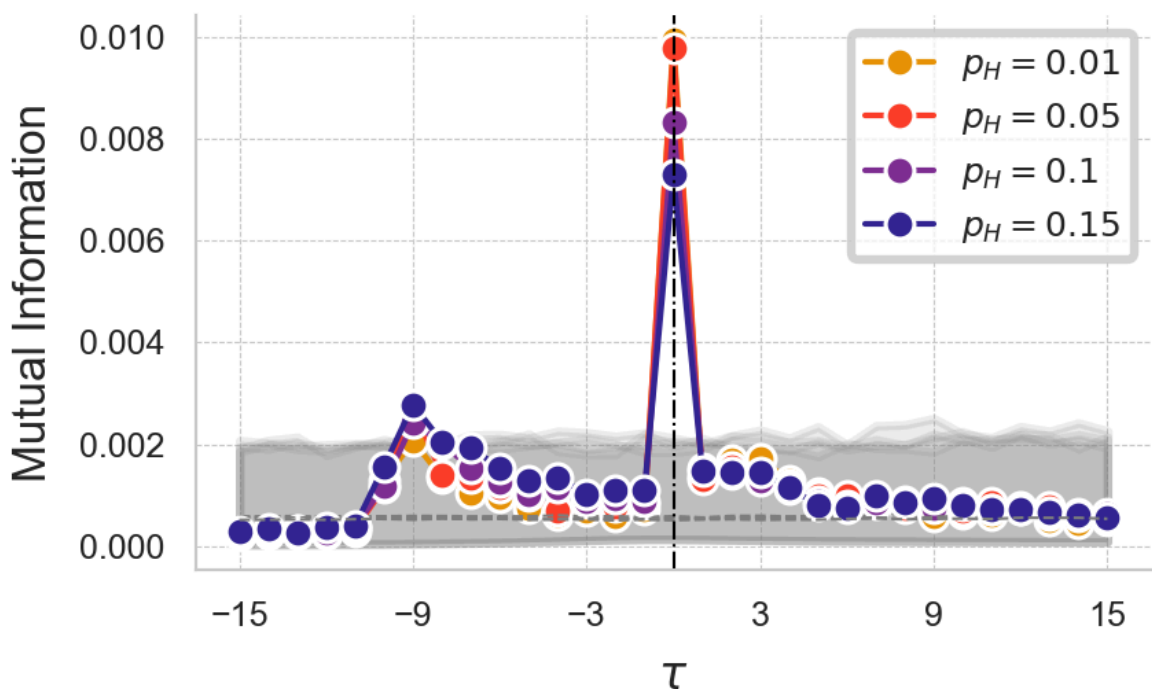
        fig, ax = plt.subplots(figsize=(3.3, 2.2), dpi=300)
        for i, pg in enumerate(pg_values):
            sns.lineplot(x=lags_mi, y=all_mi_pg[i], marker='o', label=f'$p_H={pg}$',
                        color=palette_mi[i], linewidth=1, markersize=5, ax=ax)
            ax.plot(lags_mi, all_mi_sm[i], color='gray', linestyle='--', linewidth=0.5)
```

```

ax.fill_between(lags_mi, all_mi_sl[i], all_mi_su[i], color='gray', alpha=0.5)

ax.set_ylabel('Mutual Information', fontsize=8)
ax.set_xlabel(r'$\tau$', fontsize=8)
ax.set_xticks(np.arange(-max_lag, max_lag + 1, 6))
ax.tick_params(axis='both', labelsize=6, length=2, width=0.5)
ax.axvline(0, color='black', linestyle='-.', linewidth=0.5)
ax.grid(True, linestyle='--', linewidth=0.25)
sns.despine(ax=ax)
ax.spines['left'].set_linewidth(0.5)
ax.spines['bottom'].set_linewidth(0.5)
ax.legend(fontsize=6, loc='upper right')
plt.tight_layout()
plt.savefig(f'{out_plot_directory}/{num_agents}/cdmx_mutual_information.pdf')
plt.show()

```



4. Barcodes — Visualización de eventos discretos

(Suplementario del paper)

```

In [11]: # — Parámetros para los barcodes —————
p_g     = 0.15
h       = 20
rep     = 0
delta   = 15

chi_hat_file = f'{intermediate_folder}/pg_{p_g}_h_{h}_rep_{rep}_delta_{delta}'
gamma_file   = f'{intermediate_folder}/pg_{p_g}_h_{h}_rep_{rep}_delta_{delta}'

chi_hat_df = pd.read_csv(chi_hat_file, index_col=0)
gamma_df   = pd.read_csv(gamma_file,   index_col=0)

```



```

chi_hat_df = chi_hat_df.fillna({c: {i: 0 for i in range(h+1)} for c in chi_hat_df.columns})
gamma_df = gamma_df.fillna({c: {i: 0 for i in range(h+1)} for c in gamma_df.columns})

# CAMBIO: columna de ejemplo '(5, 6)' → válida para 5×5 máx es (4,4)
# Buscar automáticamente una columna con señal
cols_con_senal = [c for c in chi_hat_df.columns
                  if gamma_df[c].max() > 0 or chi_hat_df[c].max() > 0]
column = cols_con_senal[0] if cols_con_senal else chi_hat_df.columns[0]
print(f"Columna seleccionada: {column}")

chi_hat_series = chi_hat_df[column].values
gamma_series = gamma_df[column].values
chi_hat_bin = create_chi_hat_bin(chi_hat_series)
gamma_bin = create_gamma_bin(gamma_series, find_peaks_custom)

```

Columna seleccionada: (0, 0)

```

In [12]: def plot_triple_lines_and_barcodes(chi_hat_series, gamma_series, chi_hat_bin,
                                             fig_width=3.3, fig_height=6.6):
    fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(fig_width, fig_height),
                                         dpi=400, sharex=True)

    # Panel superior: series continuas
    ax1.plot(chi_hat_series, label=r"$\mathcal{G}_{\text{bin}}$", color='#2A2E34',
             linewidth=0.8, marker='s', markersize=1.4, markeredgewidth=0.55)
    ax1.plot(gamma_series, label=r"$\mathcal{G}_{\text{net}}$", color='#eeba2c',
             linewidth=0.8, marker='o', markersize=1.4, markeredgewidth=0.55)
    sns.despine(ax=ax1)
    ax1.grid(True, linestyle="--", linewidth=0.25)
    ax1.set_yticks(np.arange(0, 1.1, 0.2))
    ax1.set_yticklabels([f"{y:.1f}" for y in np.arange(0, 1.1, 0.2)], fontsize=10)
    ax1.spines['bottom'].set_color('black'); ax1.spines['left'].set_color('black')
    ax1.spines['bottom'].set_linewidth(0.5); ax1.spines['left'].set_linewidth(0.5)
    ax1.tick_params(axis='both', labelsize=8, length=2, width=0.6)
    ax1.set_ylim(-0.25, 1.1)
    ax1.set_title(r"$\mathcal{G}_{\text{bin}}$ y $\mathcal{G}_{\text{net}}$", fontsize=10, fontweight='bold')
    leg = ax1.legend(title='', fontsize=7, frameon=True)
    leg.get_frame().set_edgecolor('darkgrey'); leg.get_frame().set_linewidth(1)

    # Barcodes
    for ax, series, name, color in zip(
        [ax2, ax3],
        [gamma_bin, chi_hat_bin],
        [r"$\widetilde{\mathcal{G}}_{\text{net}}$", r"$\widetilde{\mathcal{G}}_{\text{bin}}$"],
        ['#eeba2c', '#2A2E34']
    ):
        for idx in np.where(series)[0]:
            ax.axvline(x=idx, color='white', linewidth=2.5)
            ax.axvline(x=idx, color=color, linewidth=1.5)
        sns.despine(ax=ax)
        ax.grid(True, linestyle="--", linewidth=0.15, alpha=0.6)
        ax.yaxis.set_ticks([]); ax.yaxis.set_ticklabels([])
        ax.spines['bottom'].set_color('black'); ax.spines['left'].set_color('black')
        ax.spines['bottom'].set_linewidth(0.5); ax.spines['left'].set_linewidth(0.5)
        ax.tick_params(axis='x', labelsize=8, length=2, width=0.6)
        ax.set_ylim(-0.25, 1.1)

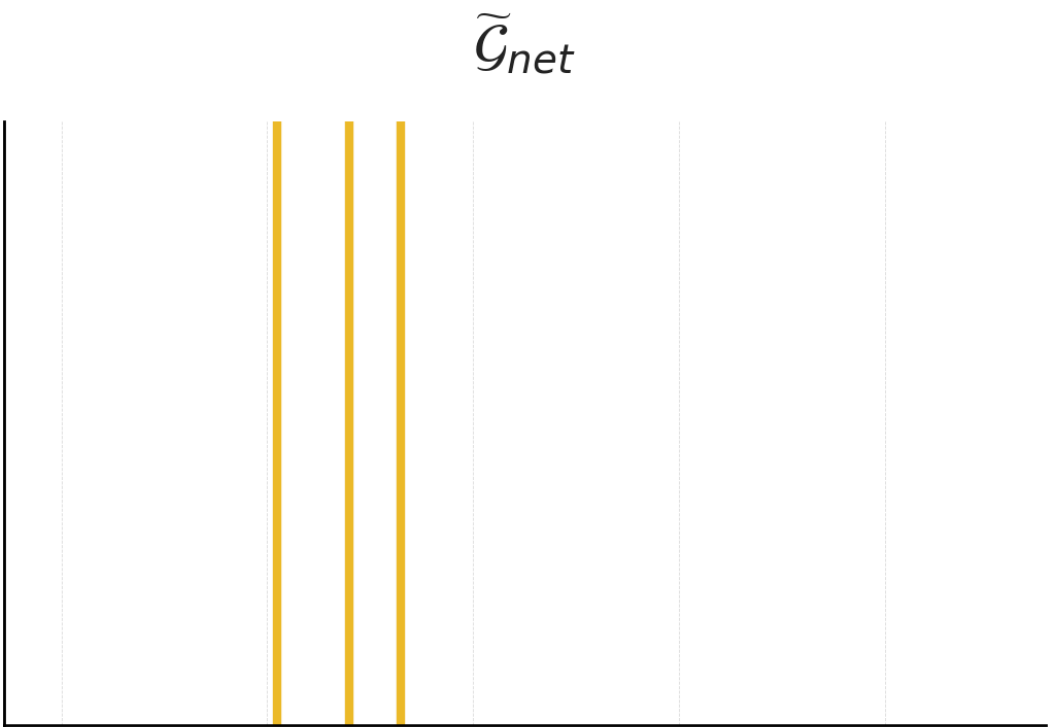
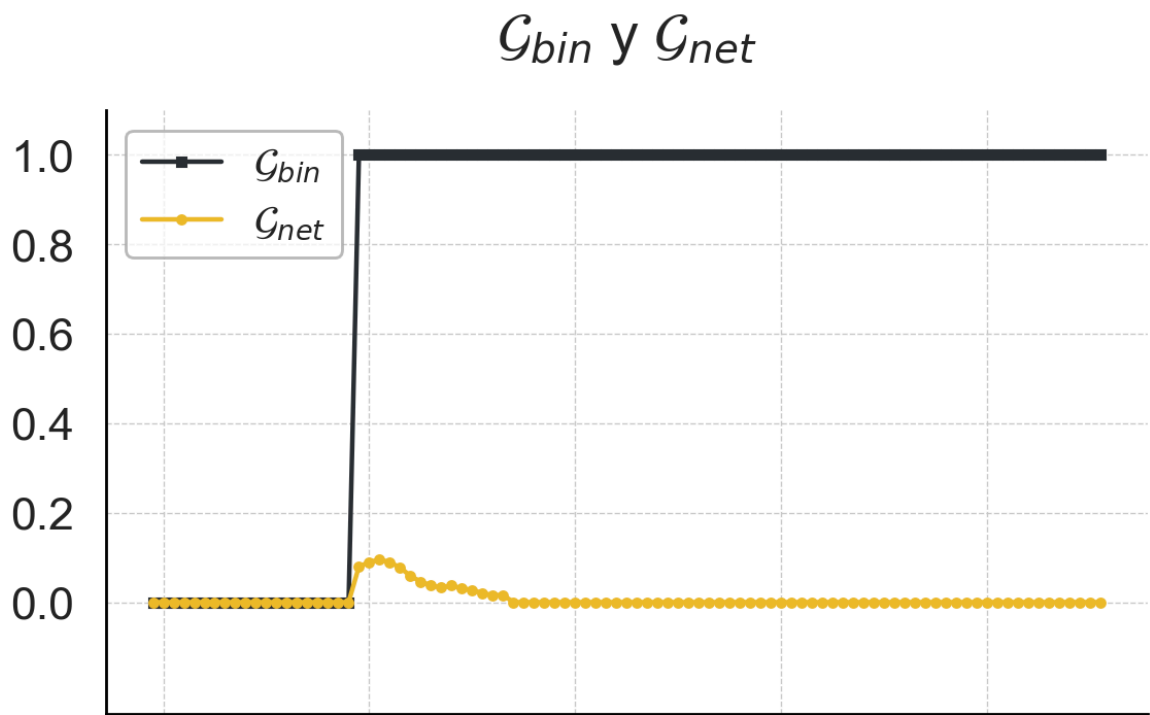
```

```
ax.set_title(name, fontsize=10, pad=10)

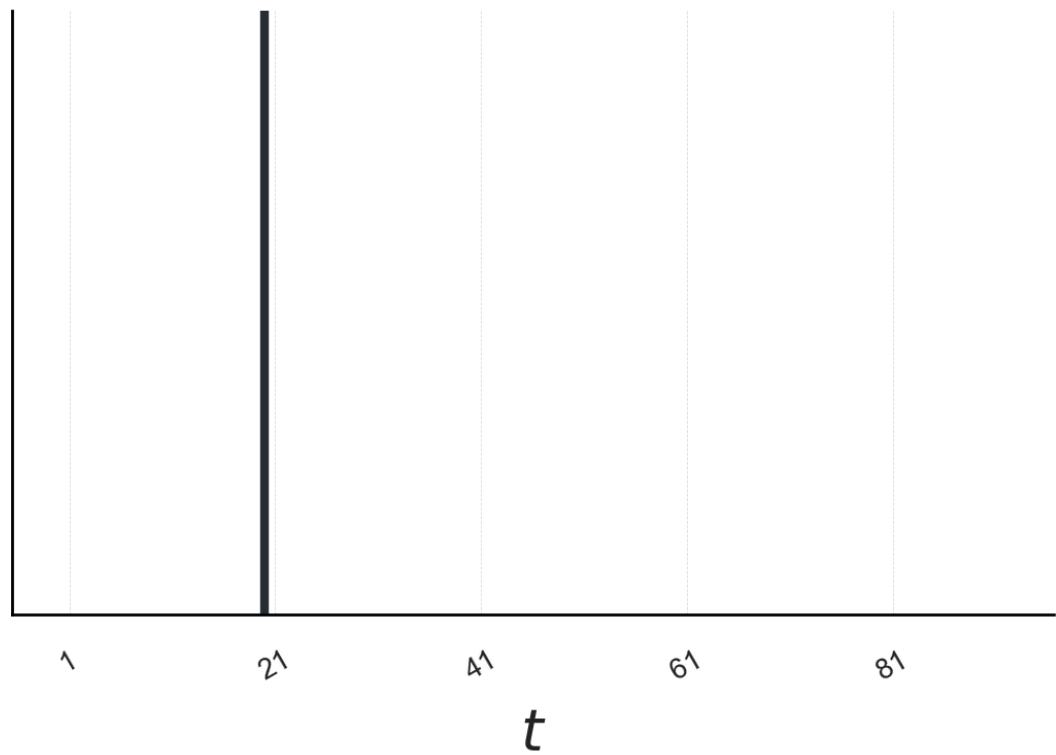
for ax in [ax1, ax2, ax3]:
    ax.set_xticks(np.arange(1, len(chi_hat_series), 20))
    ax.set_xticklabels(np.arange(1, len(chi_hat_series), 20), fontsize=5)
ax3.set_xlabel(r"$t$", fontsize=10)
plt.tight_layout()
return fig, (ax1, ax2, ax3)

fig, axes = plot_triple_lines_and_barcodes(chi_hat_series, gamma_series, chi
fig.suptitle(f"CDMX - celda {column}, p_H={p_g}, rep={rep}", fontsize=7, y=1
fig.savefig(f"{out_plot_directory}/{num_agents}/cdmx_barcodes_pg{p_g}_rep{re
plt.show()
```

CDMX — celda (0, 0), p_H=0.15, rep=0



$\tilde{\mathcal{G}}_{bin}$



In []: